AI log

Generative AI was used throughout the development of this app. I used it mainly to save time on boilerplate, tests and side scripts, so I could spend that time on the core logic and design choices of the app. The main decisions were mine, and I checked the results with benchmarks and live chats. How much I was involved by hand differs from part to part, so I split this log into two sections.

The main purpose of this log is better transparency, and hopefully to make the process of grading/reviewing this project easier for the teachers. I tried to keep this file short, but also informative. I will be more than happy to provide extra information if needed.

1. AI-generated code with little or no manual editing

AI was used to generate supporting code where implementation details were less central to the application's core logic or where the code was used temporarily. I judged these by their behaviour and not by their implementation, because they are throwaway or support code that I wasn't going to maintain or scale, and they didn't impact the actual state of my app in any way.

Example flow:

1. I implement a handful of new features.
2. I need a new set of tests.
3. Prompt an AI with my new features, and current state of test.ts
4. AI adds tests, or invents a new test.ts
5. I run the new test and manually break the target implementation on purpose.
    - test fails, I mark it done and move on
    - test still passes: I review the code

(same with benchmarks: I need new benchmark => I instruct what needs benching => I analyze the results and plan accordingly)

List of applications:

- Files in `test/` and most files in `scripts/`.
- One-off scripts for debugging, testing prompt variations, comparing model parameters, and measuring backend performance.
- The fix for the speechstate ASR hang (see `docs/speechstate-asr-hang.md`).
- The browser interface, including its skeleton, CSS, and intake form. These were AI-generated and reviewed, with no manual additions or fixes from me. The interface was also visually based on my previous project, `mono-vowel`.

2. AI-assisted development with substantial manual involvement

AI also assisted with debugging, refactoring and refining. In these cases, I made the decisions about what to change and evaluated the implementation and results thoroughly.

Example flow:

1. I noticed that wishes kept getting lost after a suggestion.
2. With the help of an LLM I replayed 60 taste changes against the university llama3.1 and found four causes.
3. I chose the fix: split the NLU prompt by the current question under discussion (QUD).
4. I checked it with the nlu-bench (28/36 → 34/36) and a live chat with Elisa before moving on.

(The nlu-bench had 36 cases at the time. Its cases changed several times during development, so these numbers differ from the final ones in the report.)

List of applications:

- When moving from the mood-based system to wish-based system, I implemented the new approach and then used AI to help adapt selected features from the previous implementation.
- To map module dependencies during a late-stage cleanup, then decided which modules to remove, merge, or retain.
- NLU sometimes behaved unexpectedly, and while debugging it, AI helped implement grounding.
- The NLU/NLG prompts were initially written by me and subsequently refined with AI assistance.

Every change in this section was checked with the benches and a live chat with Elisa before I moved on.

Conclusion

I deemed it important to list the above because:

- Regarding section 1, since my coding-involvement was minimal, almost non-existent, it would be fairer not to take credit for the code in these modules. Instead maybe graded on the behaviour/logic of the modules. This list is complete.
- Regarding section 2, I was directly involved in every part. All output was closely monitored and reviewed before I applied it. The section 2 list isn't complete, but I hand picked those examples, because I believe they showcase a nice variety of different tasks and how the work was split between me and AI.

The basic version from my project proposal was working early on, because I built it directly on top of the SISU setup from lab 2 (visible in the first commit) and used the TMDB API as a query tool. So most of my time went into improving it.

So concluding, AI was essential to developing the application to its **current scope** within the **available time**. I used it as a development tool while directing the architecture, making technical decisions, and evaluating the resulting behaviour. By outsourcing certain scaffolding and/or supporting scripts to AI, I could steer my whole attention to the actual app and its behaviour.